# Combining Survey Encoding and Gradient Boosting for Coronary Heart Disease Classification

Nour Chebil Ismail Moudden Ziyad Boudalal
*CouscousAI — CS-433 Machine Learning, EPFL, Fall 2026*

***Abstract*—We predict self-reported coronary heart disease (MICHD) from the 2015 BRFSS health survey with NumPy only. After cleaning the survey codes, we compare the six methods seen in class, then improve on them with a Newton-method logistic regression and our own gradient-boosted trees. Their blend reaches a cross-validated F1 of $0.4436 \pm 0.0036$ and 0.455 on AIcrowd. An ablation study shows that, after basic cleaning, the gain comes from the non-linear model, not from preprocessing.**

## I. Task and data

We have 328,135 labeled respondents and 109,379 to predict, each with 321 features. Only 8.83% are positive, so always answering "no" gives 91.17% accuracy but an F1 of zero. We therefore optimize the positive-class F1, $2TP/(2TP + FP + FN)$, which is also the leaderboard metric, and tune the decision threshold of every model instead of using 0.5.

44.8% of the values are missing and 147 columns are more than half empty, mostly because the survey skips questions depending on earlier answers: *missingness is informative*. Many answers are also codes: `PHYSHLTH` = 88 means no unhealthy days, while 77 and 99 mean "don't know" and "refused" [1].

## II. Preprocessing and models

**Special codes.** "Don't know / refused" codes (7/9, 77/99, ...) become missing. From the codebook we fixed key columns by hand: "none" codes (88, 888, 555) become 0, 7 stays valid for race and employment, frequencies become times per day or month, weight and height kg and cm. A rule based on each column's largest value handles the others.

**Features for the linear models.** We drop constant columns, columns more than 80% empty and eight bookkeeping fields (dates, `SEQNO`, `_PSU`, `_STSTR`). Columns with at most 30 values, and `_STATE`, are one-hot encoded: rare levels, missing and unseen values share an "other" column, and the most frequent level gets no column (otherwise the columns sum to the bias). Numerical columns are median-imputed, clipped to their 1st–99th percentiles and standardized. We add 0/1 columns for "was missing" (one per missingness pattern, also for dropped columns up to 99% missing) and "answered none", the squares of the 10 numerical features most correlated with the label, and the 300 pairwise products of the 25 most correlated encoded columns. Every statistic and selection is learned on the training fold only.

**Methods from class.** We implemented the six methods in `implementations.py`. To choose a learning rate that does not diverge, we compute $L$, the largest eigenvalue of $X^\top X/N$ ($L = 13.35$), and use $\gamma = 1/L$ for least squares and $\gamma = 4/L$ for logistic regression. Least squares uses float64 and a pseudo-inverse when $X^\top X$ is singular.

**Newton logistic regression.** For $y_i \in \{0, 1\}$ we minimize

$$L(w) = \frac{1}{n} \sum_i \left[\log(1 + e^{x_i^\top w}) - y_i x_i^\top w\right] + 10^{-4}\|w\|_2^2, \quad (1)$$

penalizing all coefficients, including the intercept. With about 1,000 columns the Hessian is small enough to compute (in blocks of 20,000 rows), so Newton's method with a backtracking line search converges in at most 15 steps, with no step size to tune.

**Gradient-boosted trees (GBDT).** A linear model only adds effects together, but heart disease risk depends on combinations, such as being old *and* in poor health. We therefore wrote gradient boosting for the logistic loss from scratch [2], [3], [4]: each tree corrects the errors of the previous ones. Features are cut into 63 bins plus a missing bin (no imputation). A tree (depth 4) sums the gradients $g$ and Hessians $h$ per bin (`np.bincount`) and keeps the split with the largest gain $\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}$, also learning whether missing values go left or right. Leaves predict $-G/(H + \lambda)$ ($\lambda = 1$, minimum Hessian sum 5) times a learning rate of 0.1; each tree sees a random half of the rows and features.

**Blend.** The final decision combines both probabilities:

$$p = 0.7\, p_{\text{trees}} + 0.3\, p_{\text{logistic}}, \qquad \hat{y} = \begin{cases} +1 & p \geq 0.225, \\ -1 & \text{otherwise.} \end{cases} \quad (2)$$

## III. Evaluation protocol

All choices use the *same three shuffled folds* (seed 42, not stratified), with the preprocessing refitted inside each training fold so nothing leaks. Tree early stopping holds out 10% of the training fold and stops after 50 trees without improvement; the kept counts (183, 171, 216) give, $\times 1.1$, the 209 trees of the full-data refit. Thresholds are searched from 0.02 to 0.98 (step 0.005) on the pooled out-of-fold

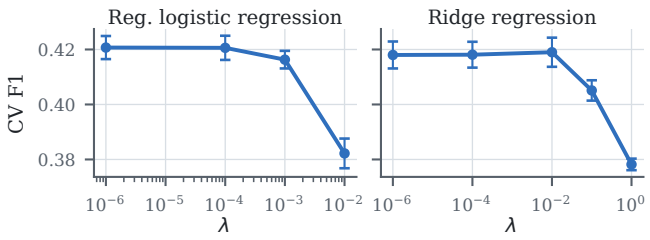



Figure 1. 3-fold CV F1 as a function of the regularization $\lambda$ (error bars: std over folds).

Table I
THE SIX METHODS ON A STRATIFIED 80/20 SPLIT (THRESHOLD TUNED ON THE VALIDATION PART).

| Method | Hyperparameters | Accuracy | F1 |
|---|---|---|---|
| MSE GD | $\gamma = 1/L$, 500 it. | 0.867 | 0.419 |
| MSE SGD | $\gamma = 10^{-3}$, 20k it. | 0.863 | 0.401 |
| Least squares | normal equations | 0.850 | 0.418 |
| Ridge regression | $\lambda = 10^{-2}$ | 0.868 | 0.420 |
| Logistic regression | $\gamma = 4/L$, 1000 it. | 0.847 | 0.419 |
| Reg. logistic | $\lambda = 10^{-6}$ | 0.847 | 0.419 |

(OOF) probabilities, and we report the mean and population std of the three fold F1 scores.

## IV. Results

The six methods all reach an F1 of 0.40–0.42 on a stratified 80/20 split (Table I). Regularization does not help: small values of $\lambda$ all give the same CV score and larger ones lower it (Fig. 1), so we keep $\lambda = 10^{-4}$.

Table II shows our progress on the common folds. Up-weighting positives helped gradient descent (0.4207 to 0.4265) but not Newton's method, so we dropped it. The trees are the only big jump, $+0.011$ over the best logistic model (about three standard deviations); depths 4 to 6 give 0.4414–0.4421. Blending adds $+0.0015$, small against the fold spread: the models make different errors (combinations and thresholds for the trees, smooth additive effects for the logistic regression), and the stronger trees get the larger weight (Fig. 2).

**Threshold.** At 0.5 the pooled OOF F1 is only 0.2517; at 0.225 the blend has precision 38.3% and recall 52.7%

Table II
3-FOLD CV F1 ON IDENTICAL FOLDS (MEAN $\pm$ STD, POOLED OOF THRESHOLD).

| Model | CV F1 | Threshold |
|---|---|---|
| Ridge, raw features (mean imputation) | 0.4092 $\pm$ 0.0021 | 0.220 |
| Logistic, gradient descent, class weights | 0.4267 $\pm$ 0.0049 | 0.725 |
| Logistic, Newton | 0.4269 $\pm$ 0.0042 | 0.205 |
| + preprocessing additions | 0.4302 $\pm$ 0.0028 | 0.195 |
| + 300 pairwise interactions | 0.4311 $\pm$ 0.0031 | 0.205 |
| GBDT (depth 4) | 0.4421 $\pm$ 0.0035 | 0.220 |
| **Blend 0.7 GBDT + 0.3 logistic** | **0.4436 $\pm$ 0.0036** | 0.225 |

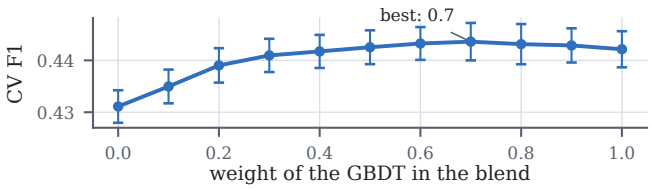



Figure 2. 3-fold CV F1 of the blend as a function of the GBDT weight (0: logistic regression with interactions only, 1: GBDT only).

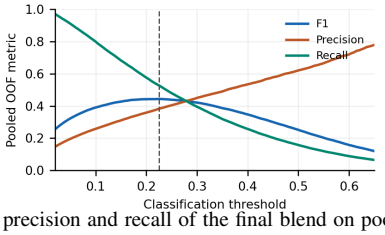



Figure 3. F1, precision and recall of the final blend on pooled out-of-fold predictions. The dashed line is the delivered threshold 0.225.

(15,272 TP, 24,602 FP, 13,703 FN; Fig. 3).

**Ablation study.** We removed one component at a time (Table III). Every change stays within the noise between folds ($\pm 0.003$), for two reasons. The codebook rules matter little because both models work around raw codes: a tree only uses the order of the values, so "88 = none" gets its own bin anyway, and in the logistic regression an untouched 88, at the end of the 0–30 range, acts as a "none" indicator. The missing-value columns help the most ($-0.0012$ without them) but are largely redundant: categorical features already have an "other" column for missing values, and trees give them their own bin.

## V. Discussion and Reproducibility

Beyond basic cleaning, the model mattered more than preprocessing. On AIcrowd (submission 338118) the blend scored 0.455 F1 and 0.886 accuracy, about 0.01 above CV, as the final models see more data. Limits: CV scores are slightly optimistic (all tuning uses the same folds), and the self-reported, cross-sectional data allow no causal or screening claim. `run.py` rebuilds the submitted CSV from saved weights.

Table III
ABLATION: CV F1 CHANGE VS. THE FINAL LOGISTIC REGRESSION (0.4302) AND GBDT (0.4421). –: COMPONENT NOT USED BY THE GBDT.

| Variant | Logistic | GBDT |
|---|---|---|
| No codebook rules (generic rule only) | $+0.0004$ | $-0.0005$ |
| Keep metadata columns | $+0.0000$ | $+0.0001$ |
| No missing indicators | $-0.0012$ | – |
| No clipping | $-0.0002$ | – |
| At most 10 values for categorical | $-0.0005$ | – |
| No "answered none" indicators | $-0.0003$ | – |
| No squared features | $+0.0000$ | – |
| No row / feature subsampling | – | $+0.0004$ |

## REFERENCES


[1] Centers for Disease Control and Prevention, "BRFSS 2015 codebook," https://www.cdc.gov/brfss/annual_data/2015/pdf/codebook15_llcp.pdf, 2016.

[2] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.